Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет	**Программной Инженерии и Компьютерной Техники**
Направление подготовки (специальность)	**Нейротехнологии и программирование**
Дисциплина	**Тестирование программного обеспечения**

ЛАБОРАТОРНАЯ РАБОТА 2

ОТЧЕТ

Выполнил студент:	***Фан Нгок Туан - 374807***
Группа:	**P3321**
Преподаватель:	**Барсуков Максим Андреевич**

г. Санкт-Петербург

2026

Вариант: 76151

1. Описание задания: 3

2. Выполнение:..... 4

3.UML..... 5

4.Описание тестового покрытия с обоснованием его выбора 5

5.Графики 10

6. Вывод: 11

1. Описание задания:

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

Введите вариант: 76151

$$\begin{cases} \left(\left(\left(\left(\left(\tan(x) - \csc(x)\right)^2\right)^3\right) - \csc(x)\right) \cdot \left(\frac{\frac{(\cot(x)\cdot\sin(x))^2}{\sec(x)}}{\cot(x)}\right)\right) + \left(\frac{(\cot(x)\cdot\sin(x))\cdot\left(\left(\cot(x)\cdot\cot(x)\right)+\left(\cos(x)-\left(\frac{\cos(x)}{\tan(x)}\right)\right)\right)}{\tan(x)}\right) & \text{if } x \leq 0 \\ \left(\left(\left(\frac{(\log_2(x)\cdot\ln(x))\cdot(\log_2(x)+\log_2(x))}{\frac{\log_{10}(x)}{\log_{10}(x)}}\right) + (\log_{10}(x) + (\log_{10}(x) + \log_{10}(x))\right) - \left(\frac{\log_5(x)}{\ln(x)}\right)\right) & \text{if } x > 0 \end{cases}$$

x <= 0 : ((((((tan(x) - csc(x)) ^ 2) ^ 3) - csc(x)) * ((((cot(x) * sin(x)) ^ 2) / sec(x)) / cot(x))) + (((cot(x) * sin(x)) * ((cot(x) * cot(x)) + (cos(x) - (cos(x) / tan(x))))) / tan(x)))

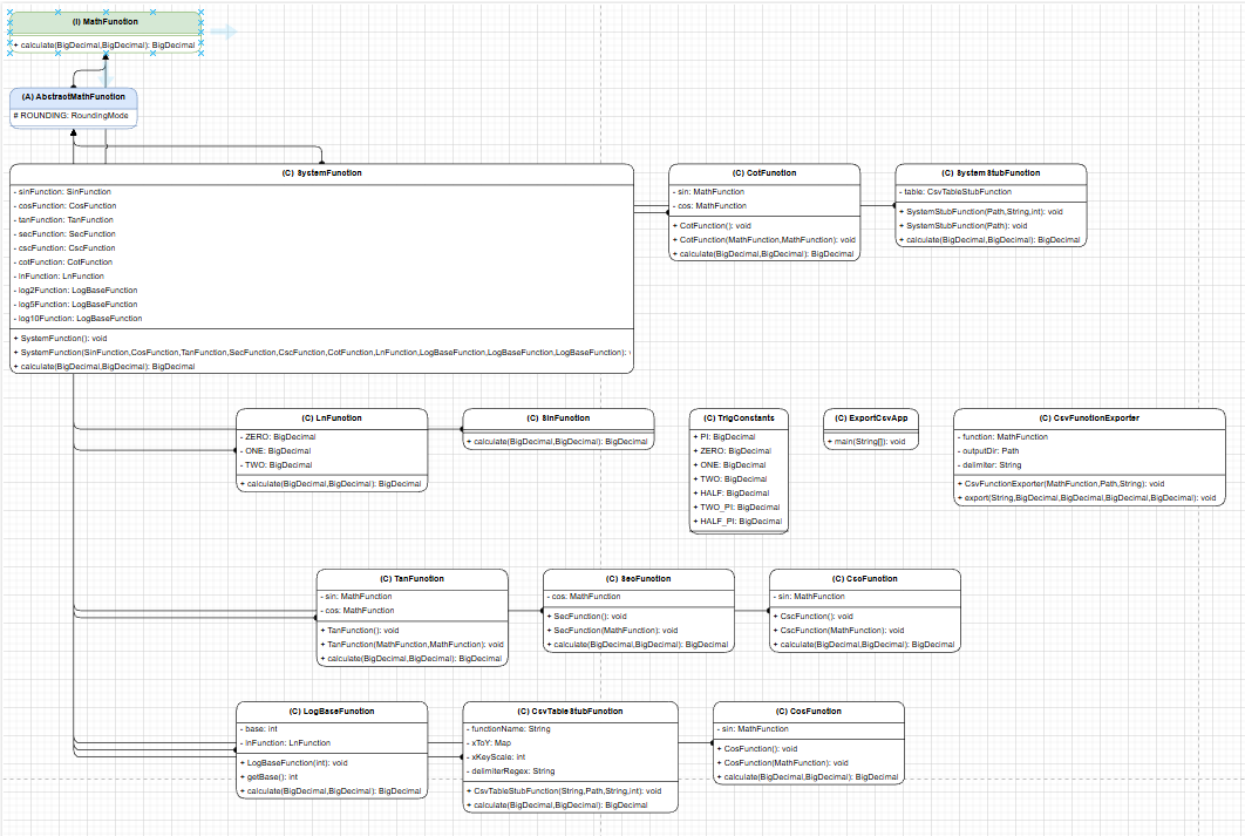
x > 0 : (((((log_2(x) * ln(x)) * (log_2(x) + log_2(x))) / (log_10(x) / log_10(x))) + (log_10(x) + (log_10(x) + log_10(x)))) - (log_5(x) / ln(x)))

2. Выполнение:

Исходный код:

https://github.com/PNT131ITMO/6.Testing/tree/main/Lab2

3. UML



4. Описание тестового покрытия с обоснованием его выбора

- **Тригонометрические функции**

При тестировании тригонометрических функций необходимо учитывать их периодичность, симметрию, а также наличие точек разрыва у производных функций (sec, csc, tan, cot). В рамках лабораторной работы базовой тригонометрической функцией является sin(x), реализованный через разложение в ряд с заданной точностью eps. Остальные функции выражены через базовую функцию, что соответствует требованиям задания.

Выделим основные классы входных значений:

1. Обычные значения (внутри периода):

   Проверка произвольных точек, не являющихся особыми (не попадающих в нули знаменателей), например:

   - $x = 0, x = \frac{\pi}{6}, x = \frac{\pi}{4}, x = \frac{\pi}{3}, x = \pi$

     На этих точках проверяется корректность вычислений относительно эталонных значений (oracle на базе Math.* в double или заранее рассчитанных значений).

2. Периодичность:
   - $\sin(x), \cos(x), \sec(x), \csc(x)$имеют период 2π

     В тестах выбираются пары значений вида xи $x + 2\pi k$(где $k \in \mathbb{Z}$), чтобы подтвердить корректность редукции аргумента и устойчивость ряда.
3. Особые точки (сингулярности / точки неопределенности):

   Для функций, содержащих деление, выделяются точки, где знаменатель стремится к нулю:
   - $\tan(x)$и $\sec(x)$не определены при $x = \frac{\pi}{2} + k\pi$
   - $\csc(x)$и $\cot(x)$не определены при $x = k\pi$

   Для этих классов точек тесты проверяют ожидаемое поведение: выбрасывание ArithmeticException при вычислении (как для sec/tan при cos(x) ≈ 0, так и для csc/cot при sin(x) ≈ 0).
4. Проверка модульной зависимости (требование задания):

   Отдельно контролируется архитектурное требование:

   cos(x) реализуется через sin(x) по формуле $\cos(x) = \sin\left(\frac{\pi}{2} - x\right)$.

   В проекте это реализовано через композицию (dependency injection):

   CosFunction использует SinFunction, что позволяет проверять корректность взаимодействия модулей и упрощает интеграционное тестирование.

- **Логарифмические функции**

При тестировании логарифмических функций необходимо учитывать область определения, особенности поведения на границах, а также то, что базовой логарифмической функцией является ln(x), реализованный через ряд с точностью eps. Функции $\log_b(x)$строятся как отношение $\ln(x)/\ln(b)$.

Основные классы входных значений:

1. Обычные значения (внутри области определения):

   Проверка нескольких значений $x > 0$, например:
   - $x = 0.1, x = 0.5, x = 2, x = 10$

     Эти точки позволяют проверить корректность вычислений как вблизи 1, так и при больших значениях.
2. Границы области определения:
   - Логарифмы определены только при $x > 0$
   - Тестируются $x = 0$и $x < 0$как некорректные аргументы

     В этих случаях ожидается выбрасывание ArithmeticException.

3. Особая точка $x = 1$:
   - $\ln(1) = 0, \log_b(1) = 0$

     С одной стороны, базовые функции корректно обрабатывают эту точку. С другой стороны, в системе (см. ниже) присутствует выражение вида log10(x)/log10(x), которое при $x = 1$становится $0/0$. Поэтому точка $x = 1$отдельно выделена как особая точка системы, приводящая к исключению.

- **Система функций**

  Система вычисляется по частям:
  - при $x \leq 0$используется тригонометрическая ветка,
  - при $x > 0$используется логарифмическая ветка.

Для интеграционного тестирования система рассматривается как верхнеуровневый модуль, зависящий от модулей тригонометрии и логарифмов. Тестовое покрытие строится так, чтобы подтвердить корректность:

1. вычисления на валидных значениях;
2. поведения на особых точках;
3. интеграции по стратегии “подключение по одному модулю”.

Сценарии тестирования:

- Обычные значения:
  - набор отрицательных значений (для ветки $x \leq 0$),
  - набор положительных значений (для ветки $x > 0$, исключая $x = 1$).
- Особые точки системы:
  - $x = 1$— система не определена из-за выражения $\log_{10}(x)/\log_{10}(x) \Rightarrow 0/0$

    Для этой точки явно проверяется выбрасывание исключения.
  - для отрицательной ветки дополнительно выделяются точки, где могут возникать сингулярности из-за $\tan(x)$, $\sec(x)$, $\csc(x)$, $\cot(x)$(например, около $k\pi, \frac{\pi}{2} + k\pi$).
- Интеграционная стратегия (“интеграция по 1 модулю”):

  В проекте реализована последовательность интеграционных тестов:

  1. система собирается полностью из табличных заглушек (CSV stubs);

2. далее по одному модулю заменяется на “реальную” реализацию (например, sin real + остальные stub, затем cos real + остальные stub и т.д.);
3. финальный сценарий — “полностью реальная система” сравнивается с oracle-реализацией на double (используя Math.sin, Math.log и т.д.) с учетом допустимой погрешности.

Такая стратегия позволяет локализовать ошибки: при падении теста становится очевидно, какой именно модуль или стык модулей нарушает корректность.

- **Общее покрытие**

  Дополнительно в проект были включены тесты следующих категорий:

  - Проверка некорректных аргументов:
    - eps вне диапазона (например, eps <= 0 или eps >= 1);
    - некорректные точки для логарифмов ($x \leq 0$);
    - точки неопределенности для функций с делением (sec, csc, tan, cot).
  - Проверка точности и допустимой погрешности:
    - для базовых функций sin(x) и ln(x) сравнение с oracle (Math.sin, Math.log) в заданной точности;
    - для системы — более широкая допустимая погрешность, так как ошибка накапливается при композиции нескольких функций.
  - Проверка использования модулей по заданию:
    - cos(x) вычисляется через sin(x) (модульная зависимость зафиксирована архитектурно),
    - log_b(x) вычисляется через ln(x).

Тестовое покрытие

Как мы можем видеть, все работает корректно:

Gradle Test Run :test

summary

22	0	0	7.255s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
SystemDomainTest	lab2.domain.SystemDomainTest	1	0	0	0.116s	100%
AllStubsIT	lab2.integration.AllStubsIT	2	0	0	0.091s	100%
CosRealIT	lab2.integration.CosRealIT	1	0	0	0.025s	100%
CotRealIT	lab2.integration.CotRealIT	1	0	0	0.022s	100%
CscRealIT	lab2.integration.CscRealIT	1	0	0	0.016s	100%
FullRealIT	lab2.integration.FullRealIT	2	0	0	0.031s	100%
LnRealIT	lab2.integration.LnRealIT	2	0	0	0.047s	100%
Log10RealIT	lab2.integration.Log10RealIT	1	0	0	0.017s	100%
Log2RealIT	lab2.integration.Log2RealIT	1	0	0	0.017s	100%
Log5RealIT	lab2.integration.Log5RealIT	1	0	0	0.024s	100%
SecRealIT	lab2.integration.SecRealIT	1	0	0	0.014s	100%
SinRealIT	lab2.integration.SinRealIT	1	0	0	0.016s	100%
TanRealIT	lab2.integration.TanRealIT	1	0	0	0.015s	100%
CsvFunctionExporterTest	lab2.io.csv.CsvFunctionExporterTest	2	0	0	0.087s	100%
SystemStubIT	lab2.system.SystemStubIT	1	0	0	0.009s	100%
LnFunctionTest	lab2.unit.LnFunctionTest	2	0	0	0.011s	100%
SinFunctionTest	lab2.unit.SinFunctionTest	1	0	0	0.005s	100%

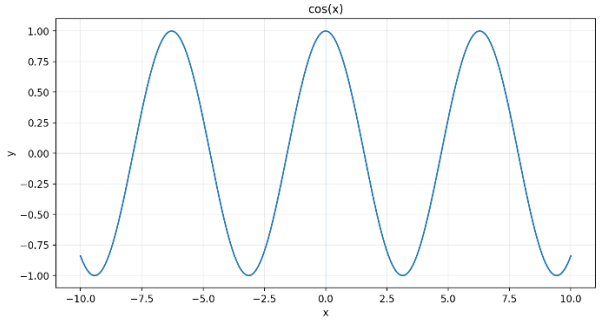
Тесты успешно проходятся.

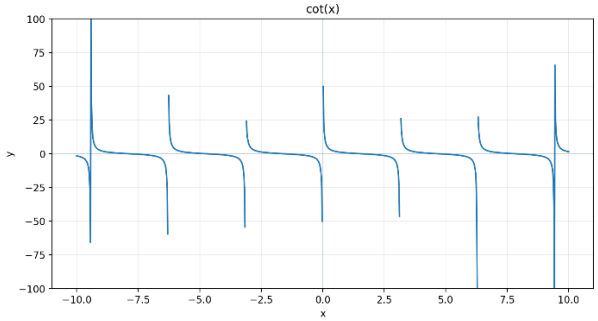
Lab2

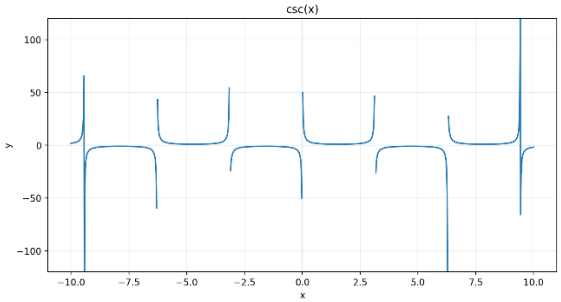
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
lab2.app		0%		0%	32	32	88	88	7	7	2	2
lab2.mathfunctions.trig		93%		68%	5	27	4	97	0	19	0	7
lab2.mathfunctions.log		87%		68%	6	14	3	40	1	6	0	2
lab2.core		70%		50%	5	11	1	11	0	6	0	1
lab2.system		98%		75%	2	11	0	79	0	7	0	2
lab2.stub		98%		78%	3	10	0	28	0	3	0	1
lab2.io.csv		98%		75%	1	4	0	22	0	2	0	1
Total	598 of 1,788	66%	71 of 118	39%	54	109	96	365	8	50	2	16

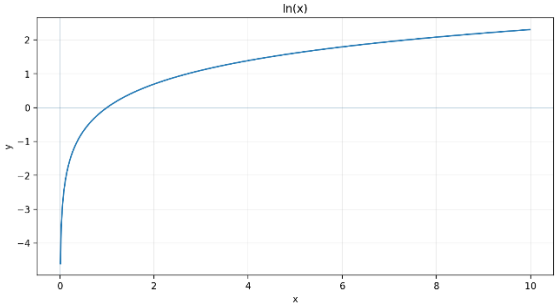
Итоговое тестовое покрытие.

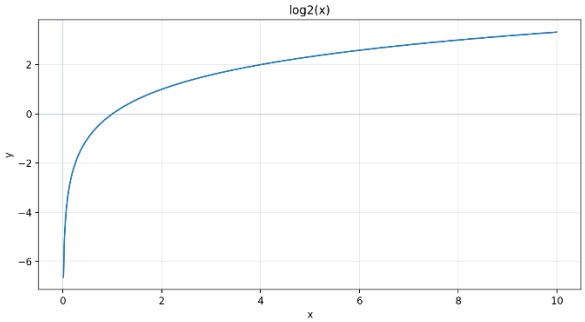
5.Графики

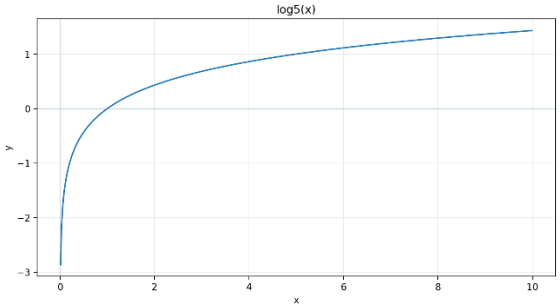

cos(x)
y
x

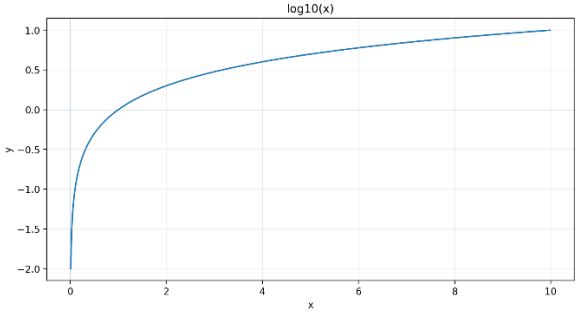

cot(x)
y
x

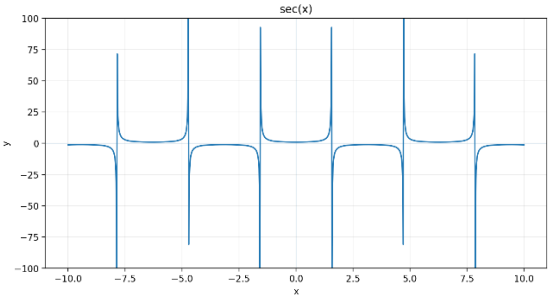

csc(x)
y
x

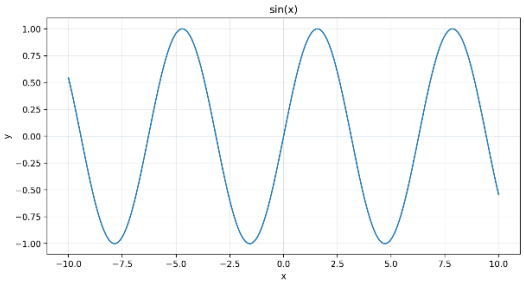

ln(x)
y
x

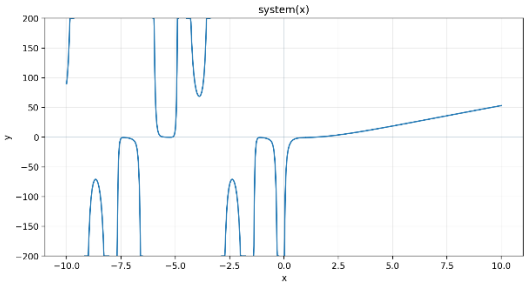

log2(x)
y
x

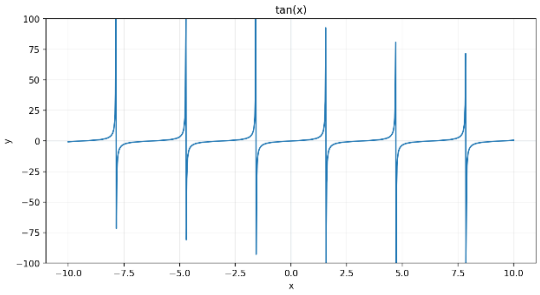

log5(x)
y
x


log10(x)
y
x


sec(x)
y
x







6. Вывод:

В ходе выполнения лабораторной работы была реализована система функций по варианту, где базовые функции sin(x) и ln(x) вычисляются разложением в ряд с заданной погрешностью eps, а остальные функции выражены через них (в том числе cos(x) через sin(x)). Проведены модульные и интеграционные тесты на JUnit 5 с учетом классов эквивалентности, граничных и особых точек, а также поэтапной интеграции модулей. CSV-выгрузки и построенные по ним графики подтвердили корректность вычислений и областей определения.